

AdaIN StyleKV: Training-Free Style Transfer with Self-Attention KV Fusion and Adaptive Instance Normalization for Diffusion Models

Junji Jiang Zhuoliang Lv Peiwen Yang

School of Computer Science and Engineering (Cyberspace Security),
University of Electronic Science and Technology of China

{junji.jiang, zhuoliang.lv, peiwen.yang}@uestc.edu.cn

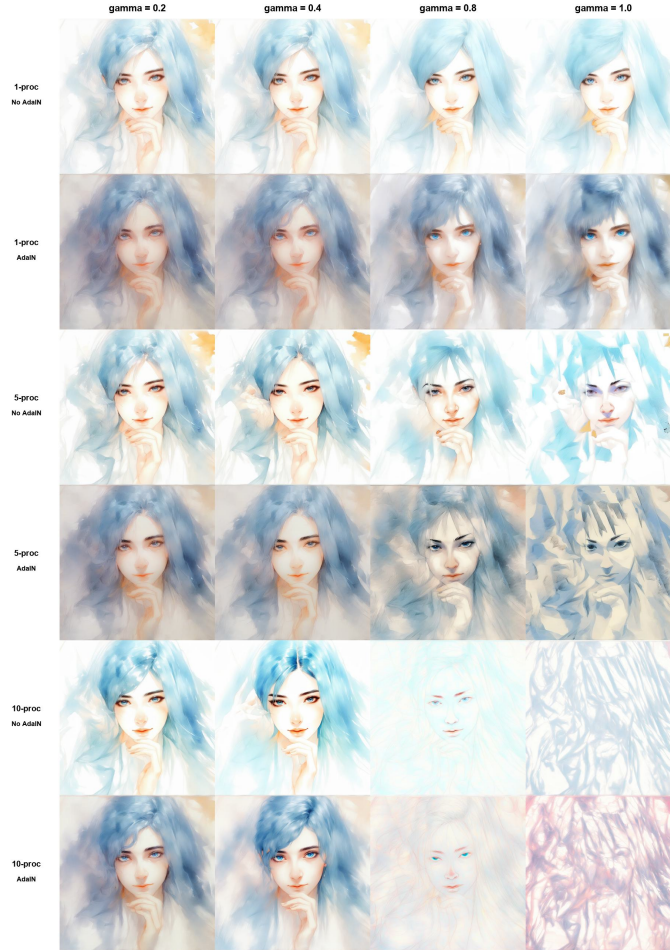


Fig. 1. Stylization results of AdaIN StyleKV. Horizontally, γ increases from 0.2 to 1.0; vertically, 1-proc, 5-proc, and 10-proc configurations are shown with and without AdaIN. The 5-proc + AdaIN configuration (our full method) achieves a favorable balance between style intensity and content fidelity near $\gamma = 0.6$.

Abstract

Artistic style transfer aims to render the visual style of a reference image onto a content image while preserving its semantic structure. Although diffusion models have significantly advanced this field, some existing methods require per-style optimization or inversion, while other training-free approaches still face challenges in structure preservation, color consistency, or intensity control. To address these issues, we propose AdaIN StyleKV, a training-free style transfer framework built upon pretrained diffusion models. The framework integrates the following key components: (1) Canny ControlNet, which preserves edge and contour structures of the content image during generation; (2) a selective self-attention KV fusion mechanism, inspired by Style Injection, which operates within the target UNet block `up_blocks.0.attentions.1` by selecting a subset of self-attention processors, caching the key-value features of the style image, and performing weighted fusion with the corresponding key-value features of the content image; and (3) a cross-modal AdaIN module that fuses embedded text features with image features at the target UNet block, enhancing color style transfer and atmospheric consistency. Component ablation, variant comparison, and parameter sensitivity analysis on MS COCO (content) and WikiArt (style) datasets demonstrate that the StyleKV configuration with 5 self-attention processors in the target block `up_blocks.0.attentions.1` achieves the lowest Art-FID (85.43) without AdaIN. Compared with the AdaIN-free StyleKV configuration, the full method reduces LPIPS from 0.629 to 0.599 after incorporating AdaIN, establishing a more stable trade-off between style injection and perceptual change. Additionally, we implement an interactive Gradio-based frontend that supports real-time adjustment of style intensity, ControlNet parameters, and AdaIN fusion coefficients.

Keywords: style transfer; diffusion models; self-attention; adaptive instance normalization; training-free; ControlNet

1. Introduction

In computer vision and computer graphics, artistic image style transfer has long been a compelling research direction. Given a content image and a style image, the goal of style transfer is to synthesize an output image that preserves the semantic structure and spatial layout of the original scene while faithfully embodying the visual style of the reference image—including texture, brushstrokes, and color palette.

The development of style transfer has undergone a paradigm evolution from non-parametric texture synthesis [1], optimization-based neural methods [2], and feedforward convolutional networks [3,4] to generative adversarial networks (GANs) [5,6,7]. Each paradigm has brought notable improvements in generation speed and transfer quality, yet inherent limitations remain: GAN-based methods are typically confined to style domains seen during training and struggle to generalize to arbitrary styles, while optimization-based methods incur heavy computational costs unsuitable for real-time applications.

In recent years, the rise of diffusion models [8,9,10] has greatly propelled progress in generative modeling. Stable Diffusion [11] and its successors leverage the latent diffusion mechanism and cross-attention modules to generate high-resolution, semantically rich images from text prompts. Inspired by this, researchers have extended diffusion models to style transfer tasks [12,13,14,15,16,17]. However, existing diffusion-based methods still face several notable challenges. First, methods such as InST [12] and StyleDiffusion [13] require gradient-based fine-tuning or textual inversion for each new style image, with per-image style adaptation often being time-consuming—limiting their applicability in large-scale or real-time scenarios. Second, some methods struggle to reliably preserve the spatial layout and semantic structure of the original image when applied to latent diffusion models, potentially altering the geometric form of the content while transferring texture. Third, although self-attention operations can effectively transfer local texture patterns, they often ignore global color statistics, leading to outputs that lack overall tonal coherence. Furthermore, most existing methods lack a mechanism for flexibly adjusting the trade-off between style intensity and content fidelity, limiting users’ creative freedom.

To address these challenges, we propose AdaIN StyleKV—a training-free style transfer framework built upon pretrained diffusion models that requires no additional fine-tuning or textual inversion, and achieves favorable stylization results under our experimental settings. This research initially began with a relatively straightforward naive AdaIN prototype: under SDXL and Canny ControlNet structural constraints, combined with InstantStyle’s effective block selection insight, AdaIN was applied to modulate text and image conditional features at target UNet blocks, with the number of target blocks configurable as 1 or 2. This prototype improved color coordination but still fell short in transferring local texture

and brushstroke patterns. Building on this observation, we further introduce a selective KV fusion mechanism inspired by Style Injection [16], forming the final AdaIN StyleKV framework. For structure preservation, the framework incorporates Canny edge detection-guided ControlNet, which effectively enhances semantic content stability by preserving the edge and contour structures of the content image during generation. For texture transfer, all StyleKV experiments prefixed with `proc` in this paper are fixed at the target block `up_blocks.0.attentions.1` for KV fusion, only varying the number of self-attention processors participating in fusion within that block; we cache the key and value features of the style image in these processors and perform weighted fusion with the corresponding features of the content image, thereby enabling effective transfer of local texture patterns. For color consistency, we take AdaIN as the methodological starting point and extend it into a cross-modal feature fusion module between text and image conditions, performing adaptive normalization fusion on the already-embedded text and image features to enhance color style transfer and overall atmospheric harmony. Furthermore, by introducing a fusion coefficient γ , the framework provides users with intuitive and continuous control over style intensity, enabling flexible trade-offs between stylization degree and content preservation.

We conduct component ablation, variant comparison, and parameter sensitivity analysis of our method on the MS COCO [18] content dataset and WikiArt [19] style dataset. Quantitative evaluation covers multiple dimensions including Art-FID [20], FID [21], and LPIPS [22]. Experimental results demonstrate that AdaIN StyleKV achieves a favorable balance between style fidelity and content preservation; notably, the StyleKV configuration with 5 self-attention processors in `up_blocks.0.attentions.1` achieves the lowest Art-FID (85.43) without AdaIN, while the full method reduces LPIPS to 0.599 after incorporating AdaIN, indicating milder perceptual change. Additionally, through multiple comparative experiments, we systematically reveal how different configurations (e.g., number of target blocks, number of KV fusion processors within `up_blocks.0.attentions.1`, fusion coefficient γ , presence or absence of the AdaIN module) affect style transfer outcomes. To enhance the method’s practicality and interactivity, we also implement an interactive Gradio-based frontend demonstration system that supports real-time adjustment of style intensity, ControlNet parameters, and AdaIN fusion coefficients.

2. Related Work

2.1. Neural Style Transfer

Neural style transfer was pioneered by Gatys et al. [2], who used Gram matrix matching of VGG [23] features through iterative optimization for style transfer. Subsequent work mainly focused on improving speed and generalization: Johnson et al. [24] introduced feedforward image transformation networks based on perceptual losses, achieving real-time style transfer; Huang and Belongie [4] proposed adaptive instance

normalization (AdaIN), enabling feedforward arbitrary style transfer. Other notable approaches include attention-based methods [25], Transformer-based methods [26], reversible flow-based methods [27], and dual style learning frameworks [28]. Although these methods significantly improved speed over the optimization-based approach of Gatys et al., they still struggle with highly abstract or non-photorealistic styles.

2.2. GAN-Based Style Transfer

Generative adversarial networks have achieved remarkable progress in unpaired image-to-image translation. CycleGAN [5] introduced cycle-consistency loss, enabling cross-domain image translation without paired data; StarGAN [6] further extended this to a multi-domain translation framework capable of handling translation among multiple target domains with a single model. In addition, high-quality generative models represented by StyleGAN [7] and its variants [29,30] provide fine-grained and interpretable latent space control, widely used for in-domain image editing and attribute manipulation. It should be noted, however, that StyleGAN is fundamentally an unconditional generative model whose design objective is not cross-domain translation; when applied to out-of-domain style transfer, it often requires additional fine-tuning or feature disentanglement strategies, making it difficult to directly achieve arbitrary style generalization comparable to translation methods such as CycleGAN.

2.3. Diffusion-Based Style Transfer

Diffusion models have become the dominant paradigm for high-quality image generation. Among them, latent diffusion models (LDM) [11] achieve efficient training and inference by compressing images into a low-dimensional latent space. For style transfer tasks, researchers have proposed various diffusion-based methods:

InST [12] employs textual inversion to embed style images into the text vector space, effectively preserving style features, but requires per-style optimization. StyleDiffusion [13] introduces CLIP-based loss functions and fine-tuning strategies to disentangle content and style. DiffStyle [14] is a training-free method operating in the H-space of diffusion models; however, applying it to Stable Diffusion may alter the semantic layout of images. Style Injection [16] achieves training-free style transfer by replacing key-value features in self-attention layers with style image features, preserving content structure to some extent but lacking explicit structural constraints akin to ControlNet and having limited color transfer capability. InstantStyle [17] discovers that only specific UNet blocks play a critical role in style transfer and achieves efficient style injection through feature embedding at these blocks. StyleStudio [31] further explores selective control of style elements in text-driven style transfer.

Table 1 summarizes the similarities and differences between our method and representative approaches. Our method starts with AdaIN feature modulation as its basis, absorbs the KV replacement idea from Style Injection and the effective block selection insight from InstantStyle, and simultaneously incorporates the structural control

Table 1. Comparison of our method with representative style transfer methods. TF: Training-Free (no fine-tuning); SP: Structure Preservation (explicit spatial constraints); CT: Color Transfer (global color handling); CI: Controllable Intensity (explicit continuous control).

Method	TF	SP	CT	CI
InST [12]	×	×	✓	×
StyleDiffusion [13]	×	×	✓	✓
DiffStyle [14]	✓	×	×	×
Style Injection [16]	✓	×	×	×
InstantStyle [17]	✓	×	×	×
AdaIN StyleKV (Ours)	✓	✓	✓	✓

capability of ControlNet. On this foundation, we replace direct substitution with controllable weighted KV fusion within the self-attention processors of the target block `up_blocks.0.attentions.1`, thereby achieving a better balance between texture transfer and color harmony.

2.4. Conditional Control in Diffusion Models

ControlNet [15] proposes a neural architecture that adds spatial conditional control (edges, depth, pose, etc.) to diffusion models using zero-convolution layers. It locks the original model and creates a trainable copy, ensuring that training does not disrupt pretrained features. We integrate the Canny edge variant of ControlNet to preserve the structural contours of the content image while performing style transfer. It is important to note that the ControlNet used must match the base model version. All experiments in this paper are based on the Stable Diffusion XL (SDXL) architecture, using its corresponding Canny ControlNet weights `diffusers/controlnet-canny-sdxl-1.0`; for the SD1.5 architecture, matching weights such as `llyasviel/control_v1lp_sd15_canny` should be used instead. SDXL is chosen as the base model to leverage its higher generation resolution and stronger semantic expressiveness.

3. Method

This section details our proposed AdaIN StyleKV method. We first introduce the overall framework, then successively present the core components: selective KV fusion, cross-modal AdaIN, and ControlNet integration.

3.1. Overall Framework

Our method is based on Stable Diffusion + ControlNet (Canny), combined with cross-modal AdaIN and selective KV fusion mechanisms. The overall framework is divided into the following five stages:

Stage 1: Image Preprocessing and Encoding. Given a content image and a style image, we first extract the Canny edge map of the content image as the spatial condition input to ControlNet. Then, the VAE encoder encodes both the content and style images into the latent space.

Stage 2: DDIM Inversion and Latent Trajectory Construction. We use the DDIM sampler to invert the content and style images separately, obtaining their latent trajectories

at each timestep, which provide input for subsequent cross-modal AdaIN fusion and progressive KV feature caching.

Stage 3: Cross-Modal AdaIN of Text and Style Conditions.

During conditional injection at each DDIM inversion timestep, rather than adopting an adapter-based approach, we apply adaptive instance normalization (AdaIN) at the target UNet blocks identified by InstantStyle to perform cross-modal fusion of the already-embedded text features and image features.

Stage 4: Style Feature Caching and KV Fusion for Content Images.

We first identify the target UNet block `up_blocks.0.attentions.1`, then cache the style image K and V features from the selected self-attention processors within that block, and fuse them with content features in the corresponding processors during the reverse diffusion generation process of the content image. At each timestep: (1) during the U-Net forward pass of the style image’s corresponding latent, record the K/V features produced by the selected self-attention processors in `up_blocks.0.attentions.1`; (2) during the content latent denoising process, for the selected self-attention processors in this block, perform weighted fusion of the currently computed content K and V with the cached style K and V; (3) use the fused K/V for self-attention computation; (4) for non-target processors, keep the original K/V unchanged.

Stage 5: Decoding and Output. After reverse diffusion completes, obtain the latent representation of the generated image. Feed it into the VAE decoder to produce the final stylized image.

Fig. 2 visually summarizes the above five-stage pipeline, clearly illustrating the interaction process between content and style images in latent space.

3.2. Selective KV Fusion

3.2.1 Review of Self-Attention Mechanism

In U-Net, the self-attention layer is computed as follows. Given features X output by a residual block, three projection matrices first yield the query Q , key K , and value V :

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V \quad (1)$$

The output is then computed via scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (2)$$

where d_k is the dimensionality of the query vectors. In standard generation, Q , K , and V all originate from features of the same image. However, in style transfer tasks, we combine the content image’s K , V with the style image’s K , V through weighted fusion. It can be approximately understood that the content-side Q helps maintain the spatial organization of the current latent, while the fused K/V influence the texture and appearance patterns retrieved by attention.

Fig. 3 illustrates the overall U-Net architecture and the distribution of self-attention modules across its layers,

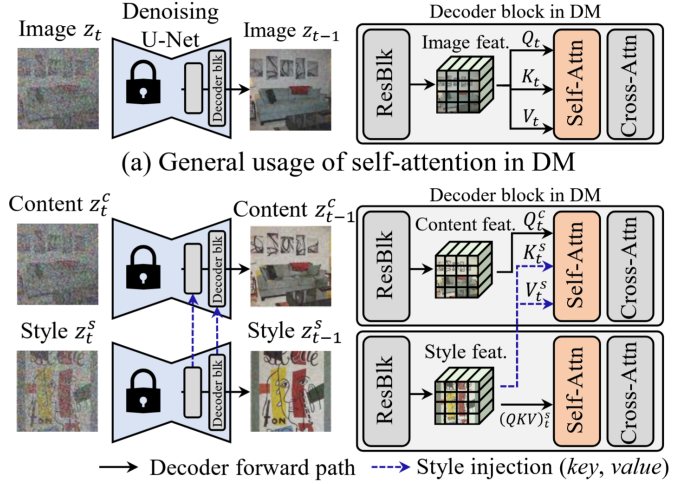


Fig. 2. Overall pipeline of AdaIN StyleKV. Given a content image and a style image, both are first encoded into the latent space via the VAE encoder, while the Canny edge map of the content image serves as the spatial condition for ControlNet. Latent trajectories of the content and style images are obtained during the DDIM inversion stage. Text and image conditions undergo cross-modal AdaIN feature fusion. During reverse diffusion, the K/V features of the style image from the selected self-attention processors in the target block `up_blocks.0.attentions.1` are cached and fused with the K/V features of the corresponding processors of the content image via weighted fusion. The stylized image is finally generated through the VAE decoder.

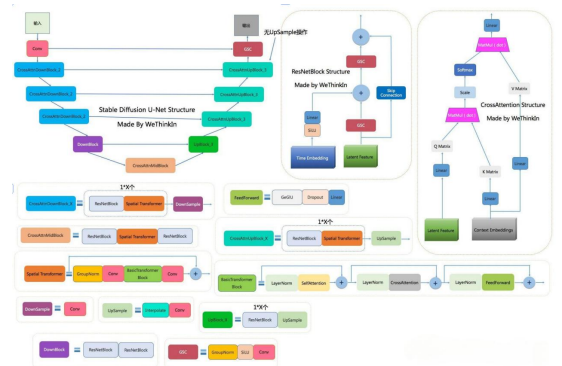


Fig. 3. Illustration of the U-Net architecture in Stable Diffusion. U-Net consists of a down-sampling encoder, a middle block, and an up-sampling decoder, each containing multiple ResNet residual blocks and self-attention layers. Our method first selects a specific decoder target block `up_blocks.0.attentions.1` based on InstantStyle, then performs KV fusion only within the target self-attention processors in that block (marked in red), while other attention processors retain their original computation.

providing structural reference for the selection of target attention processors in subsequent KV fusion.

3.2.2 Weighted KV Fusion

Inspired by Style Injection [16], we operate on the K/V within the selected self-attention processors of the target block `up_blocks.0.attentions.1`. Unlike directly replacing content image features with style image features, we adopt a weighted fusion formulation:

$$K_{\text{fused}} = (1 - \gamma)K_{\text{content}} + \gamma K_{\text{style}} \quad (3)$$

$$V_{\text{fused}} = (1 - \gamma)V_{\text{content}} + \gamma V_{\text{style}} \quad (4)$$

where $\gamma \in [0, 1]$ is the style fusion intensity coefficient. This design offers the following advantages: (1) **Adjustable**: $\gamma = 0$ fully retains content K/V, $\gamma = 1$ fully uses style K/V, with smooth transition at intermediate values; (2) **Training-Free**: style injection is achieved solely through linear interpolation, requiring no gradient optimization; (3) **Stability**: weighted fusion is smoother than direct replacement, reducing generation artifacts.

3.2.3 Selection of Target Block and Attention Processors

Following the findings of InstantStyle [17] regarding effective region selection, we first identify the UNet target blocks that are more sensitive to style transfer. All StyleKV experiments in this paper are fixed within the single target block `up_blocks.0.attentions.1`, rather than replacing K/V across multiple blocks. Within this block, a subset of self-attention processors is selected to participate in StyleKV fusion. Three processor count configurations are tested:

- **1 processor**: only 1 `attn1.processor` is selected within `up_blocks.0.attentions.1` for KV fusion
- **5 processors** (default): 5 `attn1.processors` are selected within `up_blocks.0.attentions.1` for KV fusion
- **10 processors**: 10 self-attention processors are selected within `up_blocks.0.attentions.1` for KV fusion (covering more target positions within this block)

Experiments show that the configuration with 5 self-attention processors in `up_blocks.0.attentions.1` achieves the best balance between style transfer quality and content preservation.

Table 2. Target block configurations in the early naive AdaIN prototype. 1blk/2blk indicate the number of UNet target blocks where AdaIN is applied; in contrast, StyleKV experiments are fixed at `up_blocks.0.attentions.1` and vary the number of self-attention processors.

Config	AdaIN Target UNet Block(s)	Variant
1blk	<code>up0.attn1</code>	V1: <code>adain_1blk</code>
2blk	<code>up0.attn1</code> ; <code>down2.attn1</code>	V2: <code>adain_2blk</code>

3.3. Cross-Modal AdaIN for Text and Image Conditions

In artistic style transfer, color tones and overall atmosphere constitute a significant portion of style information. However, we observe that style injection through KV fusion alone often struggles to accurately capture the global color statistics of a given style—while textures and local patterns are successfully transferred, semantic and color distribution inconsistencies between text and image conditions may still persist.

Drawing on the cross-modal AdaIN idea from StyleStudio [31], we adapt it to the StyleKV framework and combine it with selective K/V fusion and Canny structural constraints to improve color coordination and conditional consistency. Unlike methods that require training new adapters, we do not train additional adapter parameters; instead, during DDIM inversion and conditional injection, we apply AdaIN at the target UNet block for cross-modal fusion of the already-embedded text features and image features. The style image embedding in this paper is extracted by a pretrained image encoder; in the experimental implementation, we use the image encoder from the IP-Adapter configuration. AdaIN modulates the text conditional features with the channel statistics of this image embedding, aligning the text semantic guidance with the visual statistics of the reference style image.

Given text embedding features f_{text} and style image embedding features f_{style} , cross-modal AdaIN is defined as:

$$f_{\text{adain}} = \sigma(f_{\text{style}}) \left(\frac{f_{\text{text}} - \mu(f_{\text{text}})}{\sigma(f_{\text{text}})} \right) + \mu(f_{\text{style}}) \quad (5)$$

where $\mu(\cdot)$ and $\sigma(\cdot)$ denote the mean and standard deviation along the channel dimension, respectively. In the code implementation, hyperparameters α and β are further introduced, where α controls the linear mixing between the original text features and the AdaIN features, and β scales the shift term in the AdaIN branch:

$$f_{\text{adain}}^{(\beta)} = \sigma(f_{\text{style}}) \left(\frac{f_{\text{text}} - \mu(f_{\text{text}})}{\sigma(f_{\text{text}})} \right) + \beta \mu(f_{\text{style}}) \quad (6)$$

$$f_{\text{out}} = (1 - \alpha)f_{\text{text}} + \alpha f_{\text{adain}}^{(\beta)}. \quad (7)$$

Our experiments default to $\alpha = 1.0$ and $\beta = 1.0$, thus reducing to the standard AdaIN form shown in Eq. 5. This operation preserves the semantic direction in the text embedding while aligning its channel statistics to the style image embedding, thereby enhancing the consistency between text and visual style conditions without relying on additional adapter training.

3.4. ControlNet Integration

To preserve the spatial structure of the content image, we integrate Canny ControlNet. ControlNet takes the Canny edge map as conditional input, and its output is added to the U-Net skip connections and middle block. Its design advantages include: (1) zero-convolution layers ensure training stability at initialization; (2) in the original

ControlNet training paradigm, the original model parameters are frozen and only the ControlNet branch is trained; (3) plug-and-play without modifying the original model architecture. We directly use pretrained ControlNet weights matching the base model version, requiring no further training.

3.5. Conditional Generation at Inference

We do not retrain the diffusion model; instead, we inject Canny structural conditions, cross-modal AdaIN conditions, and selective KV fusion during inference of the pretrained Stable Diffusion. Given timestep t , text prompt c , Canny condition c_{canny} , and current latent z_t , the pretrained noise prediction network can be expressed as:

$$\hat{\epsilon} = \epsilon_{\theta}(z_t, t, c, c_{\text{canny}}) \quad (8)$$

where the parameters ϵ_{θ} remain frozen. AdaIN StyleKV performs modulation only on the conditional features at the target UNet block, and on the K/V features of the selected self-attention processors in `up_blocks.0.attentions.1` during inference, subsequently generating stylized images according to the five-stage pipeline described above.

4. Experiments and Results

4.1. Experimental Setup

4.1.1 Datasets

Content dataset: We use 5000 content images from the MS COCO 2017 dataset [18], each accompanied by a corresponding text prompt to assist the diffusion model in understanding the image content.

Style dataset: We use 5000 style images from the WikiArt dataset [19], covering various artistic styles such as Impressionism, Cubism, and Abstract art.

Image pairing: 5000 content images and 5000 style images are paired one-to-one in sequential order, yielding 5000 pairs in total.

Data scale: A total of 5000 stylized images are generated for quantitative evaluation.

4.1.2 Evaluation Metrics

It should be noted that all three metrics in this paper are used in the “lower is better” direction, but they measure different aspects: Art-FID aligns with the style domain, FID aligns with the content domain, and LPIPS measures per-pair perceptual change. Since style transfer inherently involves a content–style trade-off, the three metrics typically cannot be simultaneously optimal and should be interpreted jointly.

Art-FID: A comprehensive metric specifically designed for artistic image generation [20]. In our implementation, Art-FID measures the distribution distance between the set of generated images and the set of target style images, with lower values indicating closer proximity of generated results to the style domain. This is the primary evaluation metric in this paper.

FID (Fréchet Inception Distance) [21]: Measures the distribution distance between generated and content images in the InceptionV3 feature space. In this task, lower FID only indirectly reflects the overall distributional proximity of generated results to the content domain and should not be used alone as a structure preservation metric.

LPIPS (Learned Perceptual Image Patch Similarity) [22]: Computes the perceptual similarity between stylized images and content images on a per-pair basis, based on pretrained deep network feature differences. Under the same task settings, lower LPIPS generally indicates smaller perceptual change relative to the content image. Since stylization inevitably introduces perceptual changes, we interpret LPIPS jointly with FID and Art-FID rather than using it alone as a style transfer quality metric.

4.1.3 Hardware Configuration and Hyperparameters

Experiments were conducted on the AutoDL platform using a rented NVIDIA V100-32GB GPU. Hardware and hyperparameter configurations are shown in Table 3 and Table 4, respectively.

4.2. Method Variant Design

To systematically evaluate the contribution of each component, we design 7 method variants (Table 5). V1–V5 are used to evaluate the early naive AdaIN prototype and the impact

Quantitative Comparison Across Evaluation Metrics

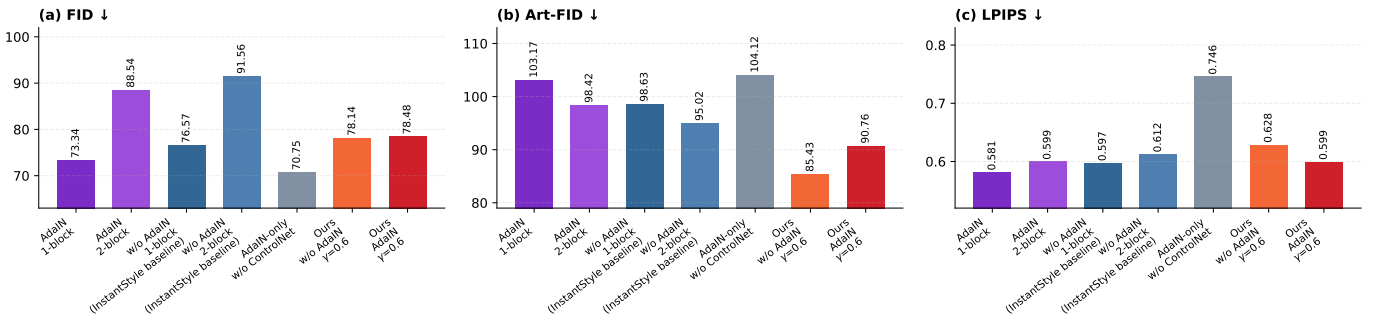


Fig. 6. Quantitative comparison of 7 method variants across FID, Art-FID, and LPIPS metrics (lower is better for all three). FID reflects distribution distance from the content domain, Art-FID reflects proximity to the target artistic style domain, and LPIPS reflects the perceptual-level image change magnitude. V6 (Ours w/o AdaIN) achieves the lowest Art-FID, indicating stronger style injection; compared with V6, the full method V7 (Ours AdaIN, $\gamma = 0.6$) reduces LPIPS, indicating milder perceptual change.

Table 3. Hardware configuration and environment.

Configuration Item	Parameter
GPU	NVIDIA V100 32GB $\times$ 1
Acceleration Library	xformers
Sampling Steps	50 steps (DDIM)
Resolution	512 $\times$ 512 (center crop)
Total Generated Images	5000

Table 4. Hyperparameter settings. $\gamma = 0.6$ is the experimentally determined favorable trade-off point (see Section 4.4 for sensitivity analysis); KV fusion defaults to 5 self-attention processors in `up_blocks.0.attentions.1`; AdaIN fusion intensity defaults to 1.0.

Parameter	Value	Description
γ	0.6	Style fusion intensity
<code>up0.attn1.procs</code>	5	KV fusion processor count
<code>adain_alpha</code>	1.0	AdaIN fusion intensity
<code>adain_beta</code>	1.0	AdaIN fusion bias
CFG scale	7.5	Classifier-free guidance
DDIM steps	50	Diffusion sampling steps

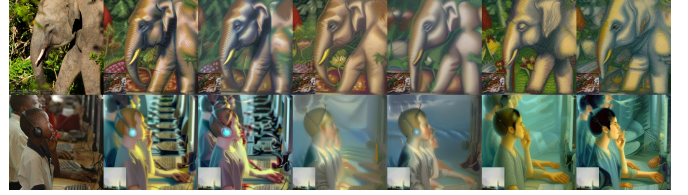
of ControlNet, where `1blk/2blk` denote the number of UNet target blocks for AdaIN (the specific blocks are listed in Table 2); V6–V7 further introduce 5-processor StyleKV fusion in `up_blocks.0.attentions.1` to compare style injection and perceptual change differences with and without AdaIN. It should be emphasized that all variants prefixed with `proc` only vary the number of processors within this block.

It is worth clarifying that in this paper, only comparisons where a single factor changes are referred to as component ablation experiments: V6 vs. V7 analyzes the impact of the AdaIN module, and V1 vs. V5 analyzes the impact of the ControlNet structural constraint. V1/V2 vs. V3/V4 primarily reflect configuration differences due to changes in the number of target blocks in the early naive AdaIN prototype; γ value variations belong to parameter sensitivity analysis; the 1-proc, 5-proc, and 10-proc results are used for qualitative observation of the impact of the number of StyleKV target processors within the same target block. Therefore, the experiments in this paper collectively comprise component ablation, method variant comparison, parameter sensitivity analysis, and configuration qualitative analysis, rather than categorizing all results as strict ablation.

Table 5. Design of 7 method variants. CtrlN = ControlNet; Scope = target blocks (blk) or processors (p); KV = KV fusion enabled.

#	Variant	CtrlN	Scope	AdaIN	γ	KV
V1	<code>adain_1blk</code>	✓	1blk	✓	—	×
V2	<code>adain_2blk</code>	✓	2blk	✓	—	×
V3	<code>noadain_1blk</code>	✓	1blk	×	—	×
V4	<code>noadain_2blk</code>	✓	2blk	×	—	×
V5	<code>adain_no_ctrl</code>	×	1blk	✓	—	×
V6	<code>proc05_no_adain</code>	✓	5p	×	0.6	✓
V7	<code>proc05_adain</code>	✓	5p	✓	0.6	✓

Fig. 4 and Fig. 5 present visual comparisons across different content–style pairings for intuitive observation of the differences among variants in style injection intensity, color coordination, and content preservation.

**Fig. 4.** Horizontal comparison of outputs from different method configurations on the “man” content image. The upper labels indicate the corresponding method configuration, enabling intuitive comparison of each configuration’s effect on style injection intensity and content preservation.**Fig. 5.** Method variant comparison on two different content–style pairings.

4.3. Quantitative Results

Table 6 presents the quantitative evaluation results of the 7 variants on 5000 generated images.

Table 6. Quantitative evaluation results of 7 variants on 5000 generated images. FID measures distribution distance from the content domain (lower indicates closer overall feature distribution to original content); Art-FID measures distribution distance from the style domain (lower indicates higher style similarity); LPIPS measures the degree of perceptual change (stylization inevitably introduces perceptual changes; moderate LPIPS values indicate effective style transfer while content remains recognizable). Bold indicates the full method (V7).

Variant Name	FID↓	Art-FID↓	LPIPS
V1 (<code>adain_1blk</code>)	73.34	103.17	0.581
V2 (<code>adain_2blk</code>)	88.54	98.42	0.599
V3 (<code>no_adain_1blk</code>)	76.57	98.63	0.597
V4 (<code>no_adain_2blk</code>)	91.56	95.02	0.612
V5 (<code>adain_no_ctrl</code>)	70.75	104.12	0.746
V6 (<code>proc05_no_adain</code>)	78.14	85.43	0.629
V7 (<code>proc05_adain</code>)	78.48	90.76	0.599

Analysis: V6 (`proc05_no_adain`) achieves the lowest Art-FID (85.43), indicating that under the StyleKV configuration with 5 self-attention processors in `up_blocks.0.attentions.1`, style texture injection is stronger without AdaIN; however, its LPIPS rises to 0.629, suggesting more pronounced perceptual change. For the full method V7 (`proc05_adain`), after incorporating AdaIN, Art-FID is 90.76—style injection is slightly weaker, but LPIPS drops to 0.599; combined with qualitative results, this indicates that AdaIN helps reduce perceptual change and may improve overall color coordination. The early naive AdaIN baselines (V1, V2) can improve color coordination, but their Art-FID remains higher than the StyleKV configuration with 5 processors in `up_blocks.0.attentions.1`, indicating that relying solely on AdaIN conditional modulation is insufficient for adequately transferring local texture and brushstroke styles. V5 (without ControlNet) has the lowest FID (70.75) but the highest LPIPS (0.746), indicating that while the overall features of generated images are close to the

content domain, local content structure is severely degraded—consistent with the expectation of lacking explicit structural constraints after removing ControlNet. V2 vs. V1 and V4 vs. V3 change the number of target blocks for naive AdaIN rather than the number of KV fusion processors; results show that more target blocks enhance the impact of conditional modulation but may also introduce more noticeable content structure changes.

To more intuitively compare the relative performance of different methods across evaluation dimensions, we convert the three metrics (FID, Art-FID, and LPIPS) into rankings (1 being best, 7 being worst), with results shown in Fig. 7. It can be observed that different variants exhibit clear trade-off patterns across the three metrics: AdaIN-only w/o ControlNet ranks highest on FID but lower on Art-FID and LPIPS, indicating that the absence of structural constraints leads to more pronounced perceptual changes; Ours w/o AdaIN ranks highest on Art-FID but lower on LPIPS; the full method Ours AdaIN $\gamma = 0.6$ outperforms the AdaIN-free version on LPIPS, reflecting AdaIN’s stabilizing effect on perceptual change and color coordination.

Ablation Rank Heatmap (lower rank is better)

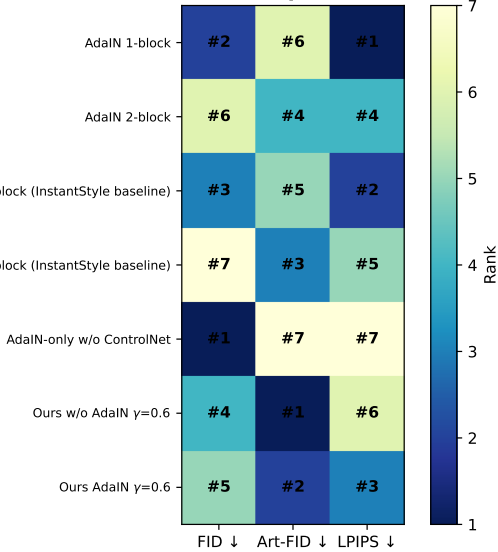


Fig. 7. Ranking heatmap of the 7 method variants across FID, Art-FID, and LPIPS metrics (lower rank indicates better performance).

Fig. 4 and Fig. 8 display visual comparison results of the variants.

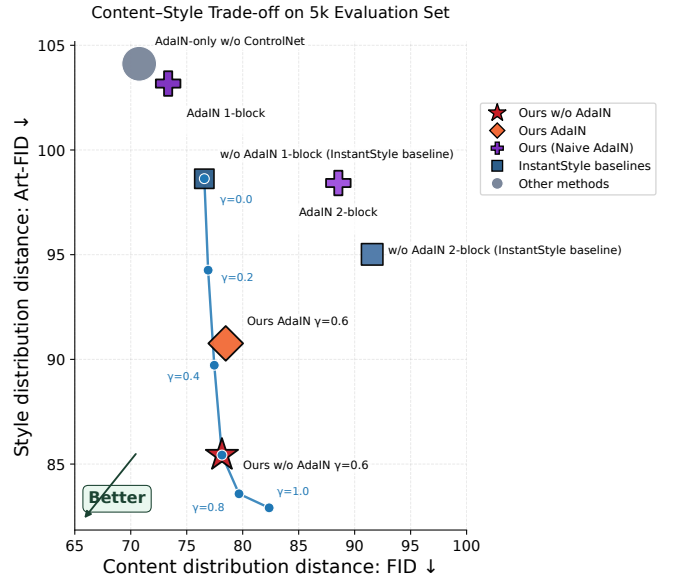


Fig. 8. Content–style trade-off comparison of method variants on FID and Art-FID.

4.4. Gamma Intensity Coefficient Sensitivity Analysis

To analyze the impact of the style fusion intensity coefficient γ on generation results, we fix the configuration of 5 self-attention processors in `up_blocks.0.attentions.1` with AdaIN and ControlNet, and vary only the value of γ , testing $\gamma \in \{0.0, 0.2, 0.4, 0.6, 0.8, 1.0\}$. Experimental results are shown in Table 7.

Table 7. Results of γ intensity coefficient sensitivity analysis (5-processor configuration in `up_blocks.0.attentions.1`, fixed AdaIN + ControlNet). $\gamma = 0$ mainly retains content features with weak style injection; $\gamma = 1$ yields the highest stylization degree but with more noticeable content change. Bold indicates our default configuration, not necessarily the single-metric optimum.

γ	FID↓	Art-FID↓	LPIPS
0.0	76.57	98.63	0.597
0.2	76.91	94.26	0.606
0.4	77.46	89.72	0.617
0.6	78.48	90.76	0.599
0.8	79.67	83.58	0.655
1.0	82.36	82.91	0.697

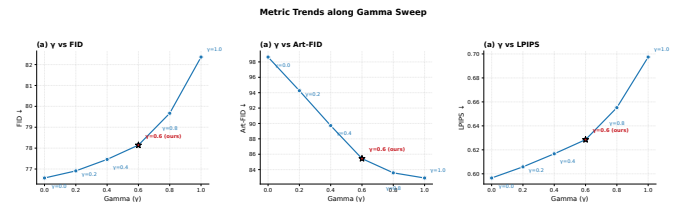


Fig. 9. Impact of γ on FID, Art-FID, and LPIPS. As γ increases, style fidelity generally improves but content change also increases; $\gamma = 0.6$ is our adopted default trade-off point.

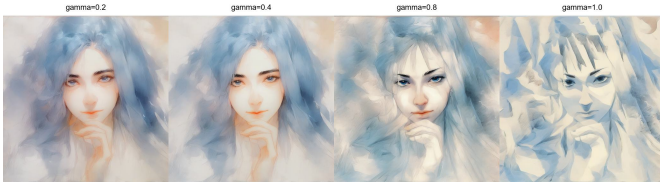


Fig. 10. Style transfer results at different γ values under the AdaIN-enabled 5-processor configuration in `up_blocks.0.attentions.1` (our method). From left to right: $\gamma = 0.2, 0.4, 0.8, 1.0$, clearly illustrating the gradual enhancement of style texture as γ increases.

Conclusion: As γ gradually increases, style texture and color expression are generally enhanced, and generated results become closer to the target artistic style distribution; at the same time, content fidelity and fine-grained structure are more susceptible to degradation. Specifically, smaller γ favors content preservation, while larger γ favors style injection. Combining quantitative metrics with visual results in Fig. 9 and Fig. 10, we select $\gamma = 0.6$ as the default trade-off point that leans toward content stability.

4.5. Qualitative Analysis of StyleKV Target Processor Count

Table 2 has already explained that `1blk/2blk` belong to target block configurations in the early naive AdaIN prototype, whereas experiments prefixed with `proc` concern the number of processors within StyleKV. These are not the same ablation dimension, so this section does not compare them as a unified quantitative ablation group. To observe the impact of the number of processors within StyleKV on style injection intensity and content preservation, we fix the target block to `up_blocks.0.attentions.1` and vary only the number of self-attention processors participating in K/V fusion within that block for qualitative visual analysis.

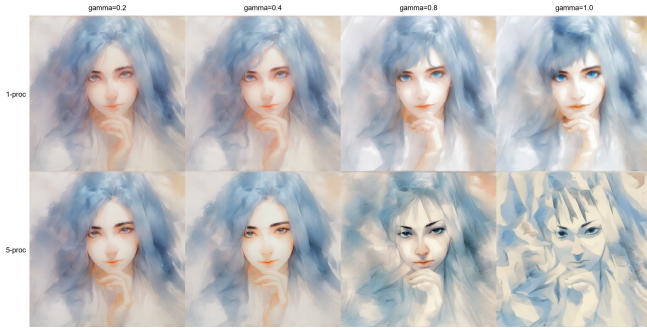


Fig. 11. Comparison of style transfer results under AdaIN-enabled 1-processor and 5-processor configurations in `up_blocks.0.attentions.1` at different γ values. Top row: 1-processor StyleKV configuration; bottom row: 5-processor StyleKV configuration (our method). More target processors yield stronger style injection (more pronounced style texture in the bottom row) while still maintaining good content structure in the shown examples.

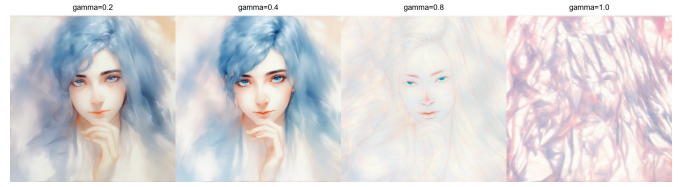


Fig. 12. Style transfer results under the AdaIN-enabled 10-processor configuration in `up_blocks.0.attentions.1` at different γ values. Compared with the 5-processor configuration in the same block, 10-processor yields stronger style injection at the same γ , but content detail preservation is slightly inferior to the 5-processor configuration.

Observation: Fig. 11 shows that within the same target block, the 5-processor configuration yields more pronounced texture transfer compared to the 1-processor configuration, especially with more sufficient style features at higher γ values. Fig. 12 further indicates that the 10-processor configuration continues to enhance stylization but is more likely to degrade facial details and local structure at high γ . Considering visual quality and computational cost together, we adopt 5 processors in `up_blocks.0.attentions.1` as the default StyleKV configuration.

4.6. Qualitative Result Analysis

4.6.1 Comparison of Different Block/Processor Configurations and AdaIN Effects

Table 8 compares the generation results of four naive AdaIN baseline configurations. Fig. 13 further illustrates the impact of the AdaIN module on color coordination under the 1-block configuration.

Table 8. Qualitative comparison of different target block counts and AdaIN configurations. Configurations using AdaIN exhibit better color coordination and more uniform style coverage; configurations without AdaIN show stronger style transfer but more noticeable content structure degradation.

Variant	#Blocks	Observation
adain_1blk	1	Concentrated style injection, noticeable texture transfer
adain_2blk	2	Uniform style coverage, better color coordination
no_adain_1blk	1	Strong style transfer, slightly inferior content preservation
no_adain_2blk	2	Enhanced style injection, more blurred structure

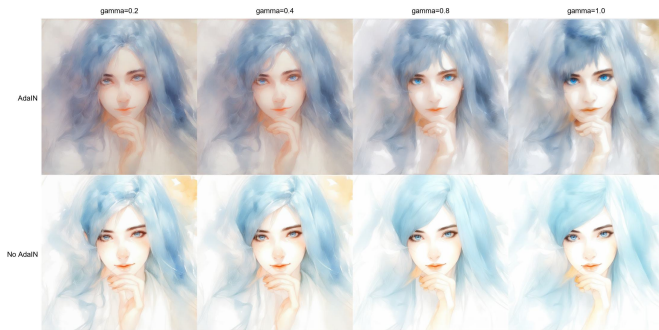


Fig. 13. Impact of the AdaIN module on style transfer (1-block configuration). Top row: results with AdaIN; bottom row: results without AdaIN. The AdaIN module significantly improves color coordination and overall atmospheric consistency, with the difference being more pronounced at higher γ values.

Fig. 1 (the teaser at the beginning of the paper) presents the overall transfer results of our method in a comprehensive comparison.

4.6.2 Impact of Prompts

In qualitative experiments, we observe that prompt quality significantly affects generation results. When using simple prompts, the model has limited content understanding and lacks detail; when using detailed prompts, the model can more accurately capture user intent, and style consistency and visual quality are generally better. This suggests that in style transfer tasks, text prompts are an important factor in guiding the diffusion model to understand content semantics.

4.7. Key Experimental Findings

4.7.1 Trade-off Between Canny Edge Map Resolution and Inference Time

The granularity of the Canny edge map directly affects generation quality and inference efficiency. Table 9 tests inference times at different Canny resolutions.

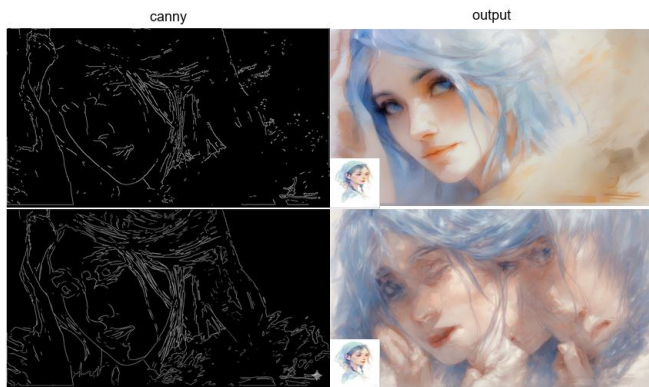


Fig. 14. Comparison of structural conditions and generation results at different Canny resolutions. Left column: Canny edge maps; right column: corresponding stylized outputs, with the reference style image embedded in the bottom-left corner of each output.

Key findings: (1) Finer Canny edge maps improve generation quality, as the model receives richer structural information; (2) however, excessively fine edge maps may cause the model to over-respond to local edges, producing outlining artifacts, fragmented textures, or unnatural local

Table 9. Canny resolution vs. inference time. Higher resolution provides richer structure but increases latency and may introduce artifacts.

Resolution	Canny Size	Time
1080×599	same as input	~12 s
5774×1536	same as input	~68 s

details; (3) inference time increases markedly with Canny resolution; (4) a balance must be struck between too fine (many artifacts, high cost) and too coarse (loss of detail). As shown in Fig. 14, the difference in Canny edge detection granularity across resolutions is significant. Based on these observations, the frontend supports dynamic adjustment of Canny edge detection parameters by the user.

4.7.2 Device Resource Constraints

Diffusion models (Stable Diffusion base model ~7–8 GB, plus VAE, ControlNet, and feature caching modules, totaling ~15–20 GB of VRAM) still pose certain challenges for consumer-grade GPUs. Even with a V100-32 GB, batch generation faces VRAM pressure and GPU resource contention. Compared with performing injection across more blocks or processors, our approach adopts selective processor injection within `up_blocks.0.attentions.1` (5 self-attention processors by default), thereby controlling the additional computation and caching overhead so that the entire pipeline can run smoothly on a V100.

5. Interactive Frontend

To enhance practical usability and user experience, we built an interactive frontend called **StyleKV Control Studio** based on the Gradio framework.

5.1. Frontend Functional Modules

(1) Image Input Module: content image upload, style image upload, automatic generation and preview of the Canny edge map.

(2) Prompt Control Module: positive prompt (describing the desired generation result), negative prompt (elements to avoid), with support for long text input.

(3) ControlNet/Canny Parameter Adjustment: Canny Low/High Threshold (edge detection sensitivity), Conditioning Scale (ControlNet control strength), Bilateral Filter parameters (edge preprocessing denoising).

(4) AdaIN Style Fusion Module: AdaIN Alpha (fusion intensity), AdaIN Beta (fusion bias), with real-time preview support.

(5) Generation Parameter Control: Steps (DDIM sampling steps, default 50), CFG Scale (classifier-free guidance, default 7.5), Seed (random seed), KV fusion intensity γ .

(6) Output Management: output directory setting, real-time generation status display, result gallery showcase.

5.2. Frontend Features and Value

The frontend supports hot parameter reloading (adjusting parameters without restarting to view results), real-time

Canny edge map preview, saving configurations as JSON files, and batch generation. The design goal is to lower the usage barrier, enabling non-expert users to understand the impact of each parameter on generation results through intuitive sliders, switches, and preview images. Fig. 15 shows the overall interface layout of the frontend.

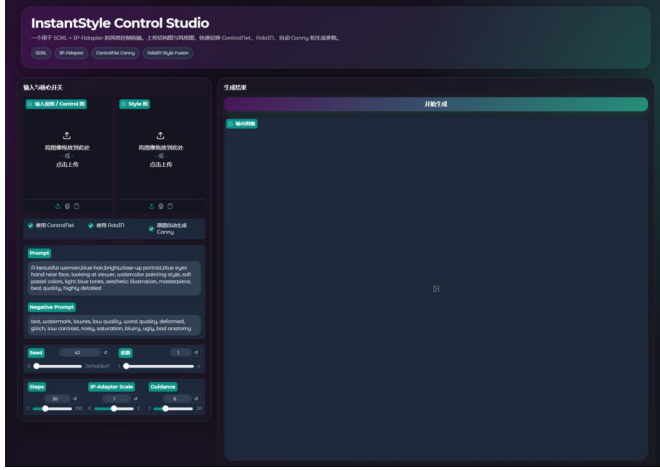


Fig. 15. Interface of the StyleKV Control Studio interactive frontend. Through the left panel, users can upload content and style images, adjust ControlNet/Canny parameters, set AdaIN fusion coefficients, and configure generation parameters; the right panel displays real-time Canny edge map previews and stylized output results.

6. Future Work

6.1. Quantitative Comparison with Existing Methods

Our current experiments focus on self-ablation to verify the effectiveness of each component and have not yet been compared with methods such as InST [12], StyleDiffusion [13], and InstantStyle [17] under equivalent conditions. We plan to conduct systematic inter-method comparisons under a unified dataset and evaluation protocol.

6.2. Adaptive Canny Edge Map Generation

Currently, Canny thresholds must be manually adjusted, and optimal values vary depending on image content. We plan to introduce better edge extraction models such as HED and RINDNet to realize adaptive edge map generation, reducing the need for manual parameter tuning.

6.3. VLM-Based Automatic Prompt Generation

Currently, prompts need to be manually written or generated with the help of external models. We plan to introduce open-source vision-language models (e.g., Qwen-VLM, LLaVA) to automatically generate high-quality prompts for style images, further lowering the usage barrier.

6.4. Cross-Architecture Adaptation and Lightweighting

Our method is validated on SDXL, and its effective attention processor selection and fusion strategy is architecture-dependent. We plan to adapt the KV fusion mechanism to

other diffusion architectures such as SD1.5 and SD3, re-identify the effective self-attention processors under each architecture, and explore lightweighting schemes such as distillation and quantization to reduce dependence on high-VRAM devices.

6.5. Incorporating Tile ControlNet

High-resolution Canny edge maps may introduce artifacts. We plan to use Tile ControlNet to mitigate artifact issues at high resolution through patch-wise processing while maintaining detail richness.

6.6. Processor-Level Contribution Analysis

Currently, we only test naive AdaIN configurations with 1 or 2 target blocks, and StyleKV configurations with 1, 5, or 10 self-attention processors in `up_blocks.0.attentions.1`. We plan to individually test the style transfer contribution of different self-attention processors in U-Net, establishing a processor-to-style mapping to provide a theoretical basis for method optimization.

6.7. Multi-Condition ControlNet Fusion

Currently, only Canny ControlNet is integrated. We plan to explore the synergistic operation of multiple ControlNets (edge + depth + pose, etc.) to further enhance content preservation capability.

7. Conclusion and Limitations

7.1. Conclusion

This paper proposes AdaIN StyleKV, a training-free style transfer method. The main contributions include:

- **Methodological innovation:** We propose a weighted fusion strategy for style image KV and content image KV, achieving precise manipulation of self-attention layer features. Drawing on the cross-modal AdaIN idea from StyleStudio [31], we adapt it to our StyleKV framework and combine it with selective K/V fusion and Canny structural constraints to improve color coordination and conditional consistency.
- **Systematic validation:** We construct 7 method variants on the COCO+WikiArt dataset and generate 5000 images for component ablation, variant comparison, and parameter sensitivity analysis. Experiments demonstrate that the full method achieves a favorable balance between style fidelity and content preservation.
- **Practical value:** We build an interactive Gradio-based frontend supporting Canny adjustment, AdaIN parameter control, real-time comparison, and other features.
- **Empirical insights:** We document key experimental findings in detail, including the importance of prompt quality and the trade-off between Canny resolution and inference time.

7.2. Limitations

Although our method achieves favorable results, the following limitations remain, as listed in Table 10. These limitations also constitute the key focus areas of the future work outlined in Section 6.

Table 10. Major limitations of our method and corresponding improvement directions.

Limitation	Description	Improvement
No external SOTA comparison	Only self-variant ablation; not compared with InST, StyleDiffusion, InstantStyle, etc.	Reproduce and compare under unified protocol
Insufficient processor ablation	Only tested few processor configs in up0.attn1	Full processor-level contribution analysis
Single ControlNet	Only Canny tested; other modalities not explored	Multi-condition ControlNet fusion
External prompt dependency	Prompts from external models; no integrated VLM	Integrate Qwen-VLM / LLaVA
Limited evaluation metrics	FID, Art-FID, LPIPS only; no perceptual/semantic coverage	Add metrics + human subjective study
High device requirements	Needs V100-class GPU; not consumer-friendly	Distillation & quantization

7.3. Future Outlook

Style transfer is a vibrant research field. With the continuous development of diffusion models, the following directions are worth sustained attention: (1) further optimization of training-free methods to achieve more precise style control while preserving content structure; (2) multi-modal style fusion, combining text descriptions, reference images, color palettes, and other conditions; (3) interpretability of style transfer—understanding the role of different U-Net layers in style transfer; (4) lightweight deployment—compressing and deploying the method on mobile devices.

Acknowledgments

We thank all teachers and fellow students who provided support and assistance to this work. We thank the AutoDL platform for providing GPU computing resources. We thank the open-source community for contributing outstanding works such as Stable Diffusion and ControlNet, which provided a solid foundation for this research.

References

- [1] A. A. Efros and T. K. Leung, “Texture synthesis by non-parametric sampling,” in *Proc. ICCV*, 1999.
- [2] L. A. Gatys, A. S. Ecker, and M. Bethge, “Image style transfer using convolutional neural networks,” in *Proc. CVPR*, 2016.
- [3] D. Ulyanov, V. Lebedev, A. Vedaldi, and V. Lempitsky, “Texture networks: Feed-forward synthesis of textures and stylized images,” in *Proc. ICML*, 2016.
- [4] X. Huang and S. Belongie, “Arbitrary style transfer in real-time with adaptive instance normalization,” in *Proc. ICCV*, 2017.
- [5] J.-Y. Zhu, T. Park, P. Isola, and A. A. Efros, “Unpaired image-to-image translation using cycle-consistent adversarial networks,” in *Proc. ICCV*, 2017.
- [6] Y. Choi, M. Choi, M. Kim, J.-W. Ha, S. Kim, and J. Choo, “StarGAN: Unified generative adversarial networks for multi-domain image-to-image translation,” in *Proc. CVPR*, 2018.
- [7] T. Karras, S. Laine, and T. Aila, “A style-based generator architecture for generative adversarial networks,” in *Proc. CVPR*, 2019.
- [8] J. Sohl-Dickstein, E. A. Weiss, N. Maheswaranathan, and S. Ganguli, “Deep unsupervised learning using nonequilibrium thermodynamics,” in *Proc. ICML*, 2015.
- [9] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” in *Proc. NeurIPS*, 2020.
- [10] Y. Song and S. Ermon, “Improved techniques for training score-based generative models,” in *Proc. NeurIPS*, 2020.
- [11] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, “High-resolution image synthesis with latent diffusion models,” in *Proc. CVPR*, 2022.
- [12] Y. Zhang, N. Huang, F. Tang, H. Huang, C. Ma, W. Dong, and C. Xu, “InST: Inversion-based style transfer with diffusion models,” in *Proc. CVPR*, 2023.
- [13] Z. Wang, L. Zhao, W. Xing, and D. Lu, “StyleDiffusion: Controllable disentangled style transfer via diffusion models,” in *Proc. ICCV*, 2023.
- [14] J. Jeong, M. Kwon, and Y. Uh, “DiffStyle: Training-free style transfer with diffusion models,” *arXiv preprint arXiv:2312.01714*, 2023.
- [15] L. Zhang, A. Rao, and M. Agrawala, “Adding conditional control to text-to-image diffusion models,” in *Proc. ICCV*, 2023.
- [16] A. Hertz, R. Mokady, J. Tenenbaum, K. Aberman, Y. Pritch, and D. Cohen-Or, “Style Injection: Training-free style transfer using self-attention,” *arXiv preprint arXiv:2312.08008*, 2023.
- [17] H. Wang, Q. Wang, X. Bai, Z. Qin, and A. Chen, “InstantStyle: Free lunch towards style-preserving in text-to-image generation,” *arXiv preprint arXiv:2404.02733*, 2024.

- [18] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, “Microsoft COCO: Common objects in context,” in *Proc. ECCV*, 2014.
- [19] WikiArt, “WikiArt Visual Art Encyclopedia.” [Online]. Available: <https://www.wikiart.org/>.
- [20] M. Wright and B. Ommer, “ArtFID: Quantitative evaluation of neural style transfer,” in *Proc. GCPR*, 2022, arXiv:2207.12280.
- [21] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, “GANs trained by a two time-scale update rule converge to a local Nash equilibrium,” in *Proc. NeurIPS*, 2017.
- [22] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, “The unreasonable effectiveness of deep features as a perceptual metric,” in *Proc. CVPR*, 2018.
- [23] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” in *Proc. ICLR*, 2015.
- [24] J. Johnson, A. Alahi, and L. Fei-Fei, “Perceptual losses for real-time style transfer and super-resolution,” in *Proc. ECCV*, 2016.
- [25] S. Liu, T. Lin, D. He, F. Li, M. Wang, X. Li, Z. Sun, Q. Li, and E. Ding, “AdaAttN: Revisit attention mechanism in arbitrary neural style transfer,” in *Proc. ICCV*, 2021.
- [26] Y. Deng, F. Tang, W. Dong, C. Ma, X. Pan, L. Wang, and C. Xu, “StyTr²: Image style transfer with transformers,” in *Proc. CVPR*, 2022.
- [27] J. An, S. Huang, Y. Song, D. Dou, W. Liu, and J. Luo, “ArtFlow: Unbiased image style transfer via reversible neural flows,” in *Proc. CVPR*, 2021.
- [28] H. Chen, L. Zhao, Z. Wang, H. Zhang, Z. Zuo, A. Li, W. Xing, and D. Lu, “DualAST: Dual style-learning networks for artistic style transfer,” in *Proc. CVPR*, 2021.
- [29] T. Karras, S. Laine, M. Aittala, J. Hellsten, J. Lehtinen, and T. Aila, “Analyzing and improving the image quality of StyleGAN,” in *Proc. CVPR*, 2020.
- [30] T. Karras, M. Aittala, S. Laine, E. Härkönen, J. Hellsten, J. Lehtinen, and T. Aila, “Alias-free generative adversarial networks,” in *Proc. NeurIPS*, 2021.
- [31] M. Lei, X. Song, B. Zhu, H. Wang, and C. Zhang, “StyleStudio: Text-driven style transfer with selective control of style elements,” in *Proc. CVPR*, 2025.

Appendix

A. Complete Data of the 7 Variants

Table 11 lists the complete experimental data of the 7 variants, consistent with Table 5 in the main text.

Table 11. Complete quantitative data of the 7 variants (consistent with Table 5 in the main text).

Variant Name	FID↓	Art-FID↓	LPIPS
V1 (adain_1blk)	73.34	103.17	0.581
V2 (adain_2blk)	88.54	98.42	0.599
V3 (no_adain_1blk)	76.57	98.63	0.597
V4 (no_adain_2blk)	91.56	95.02	0.612
V5 (adain_no_ctrl)	70.75	104.12	0.746
V6 (proc05_no_adain)	78.14	85.43	0.629
V7 (proc05_adain)	78.48	90.76	0.599

B. Complete Gamma Ablation Experiment Data

Table 12 lists the complete data of the γ ablation experiment.

Table 12. Complete γ ablation experiment data (5-processor configuration in `up_blocks.0.attentions.1` + AdaIN + ControlNet, consistent with Table 6 in the main text).

γ	FID↓	Art-FID↓	LPIPS
0.0	76.57	98.63	0.597
0.2	76.91	94.26	0.606
0.4	77.46	89.72	0.617
0.6	78.48	90.76	0.599
0.8	79.67	83.58	0.655
1.0	82.36	82.91	0.697